From Coverage to Causes: Data-Centric Fuzzing for JavaScript Engines

CSC 890: Written Preliminary Exam

Kishan Kumar Ganguly

Department of Computer Science
North Carolina State University

Advisor: Dr. Timothy Menzies

January 15, 2026

Outline

1. Introduction & Motivation
2. Literature Review
3. Our Approach
4. Phase 1 & 2: Data & Static Features
5. Phase 3: Dynamic Features
6. Phase 4 & 5: Feature Extraction & Model Training
7. Experimental Setup & Validation
8. Results
9. Key Insights & Future Directions
10. Conclusion

Why This Work Matters

Target Users: JavaScript (JS) Engine Developers & Testers

Current Scope: V8 - 75% browser share (single bug = huge impact) [*StatCounter, '25*]

Problems:

- Weeks of fuzzing, no crashes
- Enormous search space [*Jiming Wang et al., USENIX'24*; *Groß et al., NDSS'23*]
- No historical bug guidance → coverage chases low-risk paths

Our Approach:

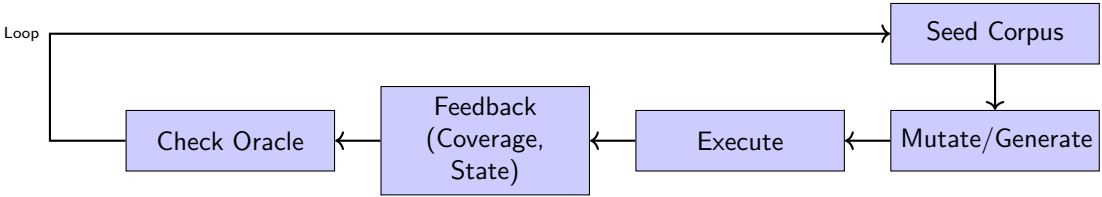
- LLM-mined bug patterns (features) → high-risk fuzzing guidance
- **41 features** (75% cut) + **5 lightweight runtime hooks** (from hundreds)
- 85%+ precision, <1% false alarms
- Found crashes as early as 5 hours

"Is this path new?" → "Does this code look dangerous?"

What is Fuzzing?

Definition

Fuzzing (or Fuzz Testing) is an automated, feedback-driven testing technique that generates and feeds large volumes of inputs to find bugs or vulnerabilities [*Miller, Fredriksen, and So, '90*; *J. Li, Zhao, and Chao Zhang, Cybersecurity'18*; *Zhu et al., CSUR'22*; *Manès et al., TSE'19*].

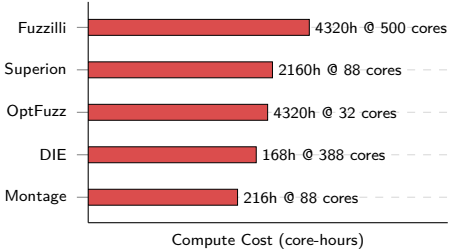


The Problem: Fuzzing JavaScript Engines is Computationally Expensive

Why This is Hard:

- Enormous search space: billions–trillions of programs [*Jiming Wang et al., USENIX'24*]
- Rare triggers: occur only under dynamic engine conditions [*Groß et al., NDSS'23*]
- Constant evolution of attack surface [*Junjie Wang, Ma, et al., TDSC'25*]

Current State: Fuzzing is Expensive



The Problem: State-of-the-art Fuzzers are Inefficient

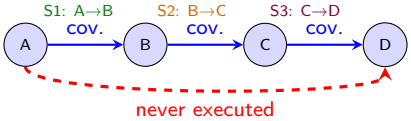
State-of-the-art fuzzers waste resources exploring vast amounts of **irrelevant, low-risk** program space.

Why:

- **Coverage-guided fuzzers** exhaustively explores new paths [*Jiming Wang et al., USENIX'24*; *J. Li, Zhao, and Chao Zhang, Cybersecurity'18*]
- **Heuristic-based fuzzers** grow inefficient over time [*Junjie Wang, Ma, et al., TDSC'25*]
- **Correctness-driven fuzzers** still explores low-risk inputs [*Lee et al., USENIX'20*]
- **Runtime/Dynamic Signals** are not utilized for learning high-risk behavior

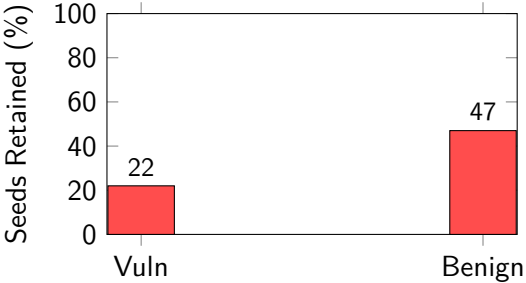
The Guidance Problem: Coverage Shows Progress, Not Vulnerability

Core Problem: Coverage $\neq$ Path Order



All edges covered individually, but full path A→B→C→D never executed [*Jiming Wang et al., USENIX'24*]

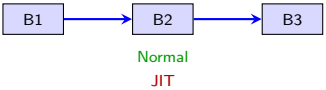
Empirical Evidence



Coverage keeps only **22%** of vulnerable seeds (160 vuln / 3000 benign)

The Invisible Dimension: Runtime Behavior

What Coverage Can't See:



Same coverage, different internal states [*Groß et al., NDSS'23*].

V8's Hidden Treasure:

- Large number of trace flags exposed
- Monitor GC, JIT, IC, maps
- Added by **developers** for debugging
- Highlight complex/error-prone areas

Observation

V8 provides rich runtime signals, but far too many for a fuzzer to use directly.

→ Fuzzers need a way to identify which signals actually indicate high-risk behavior.

Our Insight: Vulnerabilities Show Recurring Patterns

Analysis of 220 V8 Vulnerability PoCs

Same high-risk features appear repeatedly across different bugs

Static Code Patterns

- **JIT Intrinsics** — 57% of all bugs
- **Typed Arrays** — 40% of all bugs
- **Exception Handling** — 72% of all bugs

Runtime Patterns

- **GC Activity** — 57% trigger GC
- **JIT Optimization** — 61% attempt opt
- **Deoptimizations** — 48% deoptimize

21% detectable ONLY at runtime

Principal Research Question

How can we systematically extract these vulnerability patterns from historical data?

Literature Review: Current Approaches (1/2)

Coverage-Guided (15/19)

Approach: Maximize code coverage

Examples: Fuzzilli [*Groß et al., NDSS'23*], DIE [*Park et al., SP'20*], FuzzJIT [*Junjie Wang, Z. Zhang, et al., USENIX'23*] etc.

Gap: Coverage $\neq$ vulnerability

Structural (8/19)

Approach: Valid code via grammar

Examples: Superion [*Junjie Wang, Chen, et al., ICSE'19*], CodeAlchemist [*Han, Oh, and Cha, NDSS'19*], Fuzzilli [*Groß et al., NDSS'23*] etc.

Gap: Validity, not dangerousness

Heuristic-Driven (9/19)

Approach: Manual rules & domain knowledge

Examples: FuzzJIT [*Junjie Wang, Z. Zhang, et al., USENIX'23*], OptFuzz [*Jiming Wang et al., USENIX'24*], DIE [*Park et al., SP'20*] etc.

Gap: Expert-dependent, brittle

Literature Review: Current Approaches (2/2)

Dynamic Analysis (3/19)

Approach: Runtime behavior monitoring

Examples: DUMPLING [*Wachter et al., NDSS'25*], Sofi [*He et al., CCS'21*], DIE [*Park et al., SP'20*] etc.

Gap: Used for validity/oracles, predefined signals, manual instrumentation

Vulnerability Features (2/19)

Approach: Learn from past exploits

Examples: Montage [*Lee et al., USENIX'20*], PatchFuzz [*Junjie Wang, Ma, et al., TDSC'25*]

Gap: No risk modeling

Research Gap

1. No systematic extraction of vulnerability features from historical crashes
2. Dynamic runtime signals are not used for guidance

Our Contribution vs. Literature

Aspect	Prior Work	Our Work
Guidance	Coverage-based or heuristics	**Data-driven risk guidance**
Feature Source	Expert-crafted features	**LLM-derived static + dynamic features**
Dynamic Input Signals	Limited instrumentation	**Multi-source traces (5 flags)**
Vulnerability Focus	Seed corpus only	**Feature-driven model**
Adaptability	Fixed rules, non-generalizable	**Retrainable on new bugs**
Validation	Fuzzing	**Fuzzing + Time-aware cross-validation**

Research Questions

Group 1: Feature Generation Method

- **RQ1:** How can we systematically derive static and dynamic features most indicative of vulnerabilities in JS engines?

Group 2: Risk Modeling

Do these features meaningfully distinguish risky behavior?

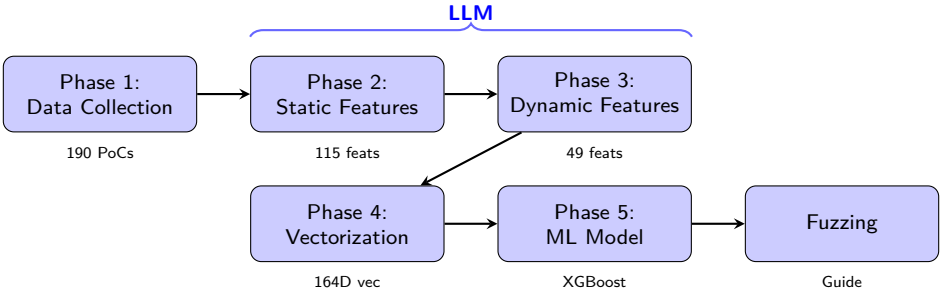
- **RQ2:** How effective is the guidance model in terms of predictive performance?
- **RQ3:** Do both static and dynamic features contribute to predictive performance?
- **RQ4:** What is the smallest feature set beyond which added features provide diminishing returns?

Group 3: Practical Impact

Do these signals actually improve fuzzing in practice?

- **RQ5:** Can the feature-guided fuzzing process discover real-world bugs?
- **RQ6:** What is the runtime overhead of collecting and using these features?

Method Overview



Core Features:

- **LLM-guided** feature generation
- Iterative refinement
- Minimal features

Final Result:

- 164 features mined $\rightarrow$ reducible to 41
- Only 5 trace flags needed
- 85%+ precision
- $<$ 1% false alarms

Phase 1: Data Collection

Positive Samples (Vulnerable)

- **Query (Chromium)**

```
componentid:(1456800|1456566|1456959|1456177|1456567|1456490)
status:(fixed|verified)
type:vulnerability
created:2019-01-01..2025-06-30
```

- Auto-reproduction (jsvu + monthly V8 versions)
- Crash detected via trace keywords
- **220 → 195 → 190 valid PoCs**
- 30 for feature design, 160 for training

Negative Samples (Benign)

- V8 mjsunit (time-matched)
- Filter out: `regress`, `bug`, `crash`
- **8.5k non-crashing tests**

Final Dataset

- 190 positive vs. 8.5k negative
- Ratio ≈ **1:53**
- Used in walk-forward CV

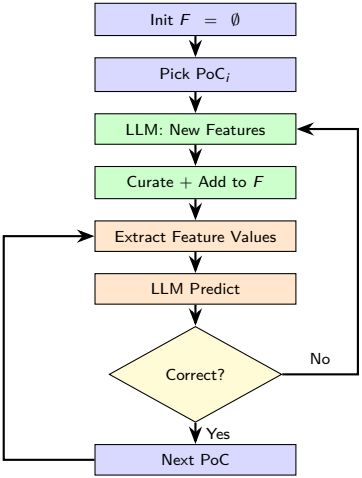
Phase 2: LLM-Guided Static Feature Generation

Process

- Input: 30 PoCs + bug report
- Output: Regex-based static features
- Up to 100 refinement iterations

LLM Prompt Structure [*Wei et al., NEURIPS'22*; *OpenAI, '24*; *Lu et al., JSS'24*]

- **Identification:** You are a JS engine vulnerability analyst
- **Task:** extract generalizable static patterns
- **Constraints:** avoid overfitting; must generalize
- **Inputs:** PoC + bug report
- **Output:** return a JSON list: name, description, regex, rationale



Result: 115 Static Features

Phase 3: Dynamic Feature Extraction

Challenge

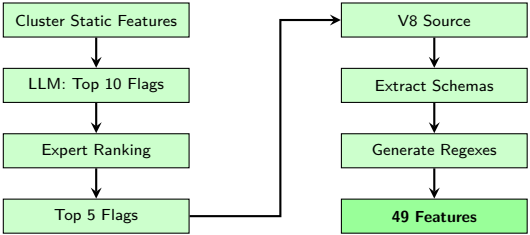
V8 has **hundreds of trace flags** → Which are vulnerability-relevant?

LLM-Guided Flag Selection:

1. Cluster static features
2. Input: Clusters + V8 flag help text
3. Output: Top 10 LLM recommended flags
4. Expert ranking + tie-breaking
5. **Top 5 flags selected**

Automated Feature Generation:

1. Extract trace schemas from V8 source
2. Generate regex-based features
3. **49 dynamic features**



Static and Dynamic Feature Examples

Static Feature Examples

- `for\s*\(.*<.*[0-9]{3,}` → Deep loops
- `function.*\{[.^}]*\1\s*\(` → Recursion
- `%OptimizeFunctionOnNextCall` → JIT optimization
- `new\s+Uint8Array` → Typed arrays
- `try\s*\{.*try` → Nested exceptions

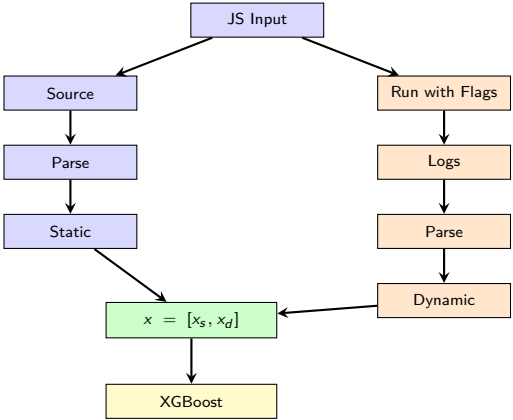
V8 Trace Flags

Flag	Purpose
`-trace-gc`	Garbage collection events
`-trace-opt`	JIT optimization decisions
`-trace-deopt`	Deoptimization events
`-trace-ic`	Inline cache transitions
`-log-maps`	Object layout changes

Dynamic Feature Examples

- GC frequency
- Failed optimizations
- IC misses
- Map deprecations
- Bailout counts

Phase 4 & 5: Feature Extraction & Building the Predictive Model



Unified Feature Vector

$$x = [x_s, x_s]$$

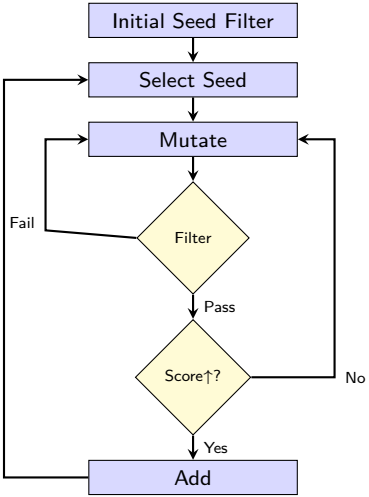
Model: XGBoost

- Handles severe class imbalance
- Captures non-linear interactions
- SOTA for tabular risk modeling [*Shwartz-Ziv and Armon, InfoFusion'22*]

Objective

- Learn a vulnerability risk function $M(x)$
- Prioritize inputs with **high-risk behavior**

Feature-Guided Fuzzing



Hybrid: **90%** feature-guided, **10%** coverage

Feature Preservation

- Static (fast) + Dynamic (selective)
- Goal: avoid losing high-risk behavior

Why Not Preserve Everything?

- Too restrictive $\rightarrow$ kills exploration

SHAP (SHapley Additive exPlanations)

- Finds features driving this seed's risk score
- Keep minimal set covering **90%** of SHAP impact

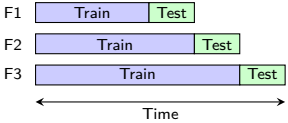
Tunable Parameter

- Preserve SHAP-important features within 1σ

Experimental Setup & Validation

Time-Aware Validation

- 4-fold CV: train on past → test on future
- Realistic temporal ordering (no leakage)



Environment

- Training: 4 cores, 16 GB RAM
- Fuzzing: 16 cores, 128 GB RAM

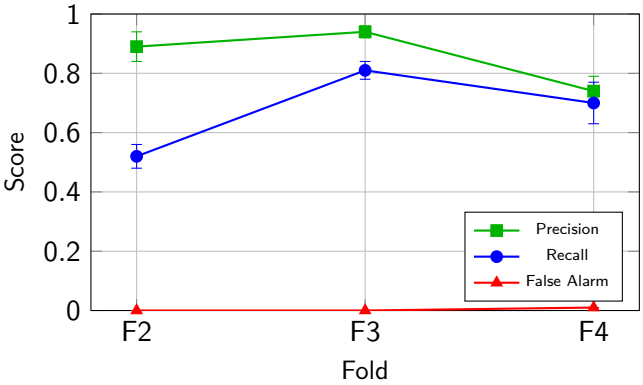
Dataset Across Folds

Metric	F1	F2	F3	F4
Positive	29	46	48	37
Negative	2064	2211	2159	2064
Ratio	1:71	1:48	1:45	1:56
Time Range	≤Nov'22	Dec'22–Aug'23	Sep'23–May'24	Jun'24–Mar'25

Metrics

- Precision $= \frac{TP}{TP+FP}$
- Recall $= \frac{TP}{TP+FN}$
- False Alarm $= \frac{FP}{FP+TN}$

RQ2: Model Performance



Key Results:

- Precision: 0.74–0.94
- Recall: 0.52–0.81
- False Alarm: $\leq$0.01

Conclusion

Model is **cautious** but **precise**. Predictions are reliable!

Why lower recall?

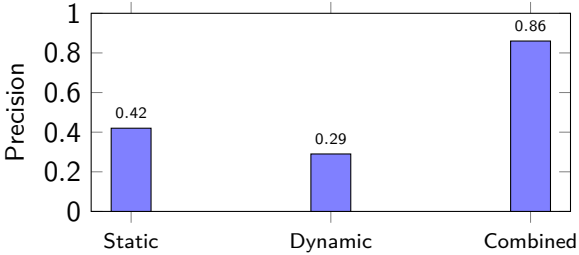
- Limited PoC diversity (30)
- Concept drift

RQ3: Static vs Dynamic Features

Method	Precision	Recall	False Alarm
Static Only	0.42 ± 0.26	0.50 ± 0.22	0.02 ± 0.01
Dynamic Only	0.29 ± 0.13	0.64 ± 0.15	0.04 ± 0.03
Combined	$\mathbf{0.86 \pm 0.10}$	$\mathbf{0.68 \pm 0.15}$	$\mathbf{0.00 \pm 0.01}$
Random	0.02 ± 0.00	0.49 ± 0.04	0.50 ± 0.01

Observations:

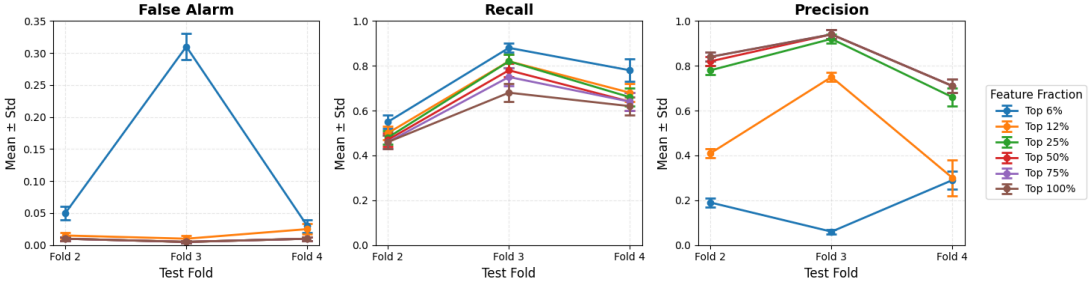
- Static: Higher precision
- Dynamic: Higher recall
- Combined: **Best of both!**
- 3× gain in precision



Conclusion

Both feature types essential with synergistic effects.

RQ4: Minimal Feature Set Analysis



Performance Metrics (Mean ± Std)

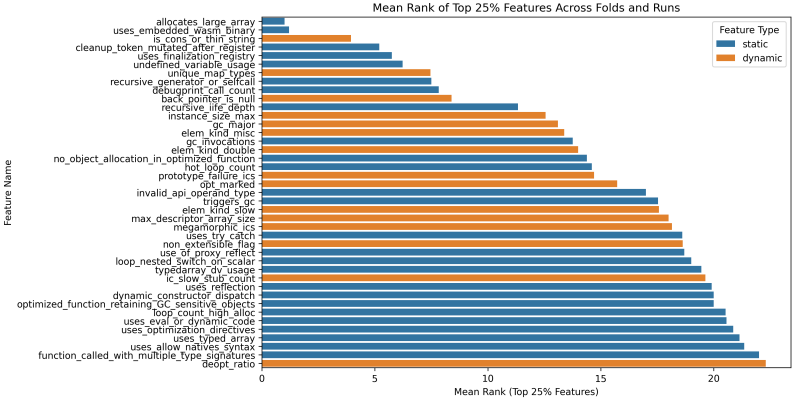
Conclusion

Top **25%** (41/164) achieve near-optimal performance!

What Gets Eliminated:

- 69% dynamic features
- 77% static features
- **75% total reduction**

Top 25% Features by Importance



Feature Composition:

- **Static**: 23% retention
- **Dynamic**: 31% retention

Conclusion

prioritizing dynamic feature collection offers more benefits in our pipeline

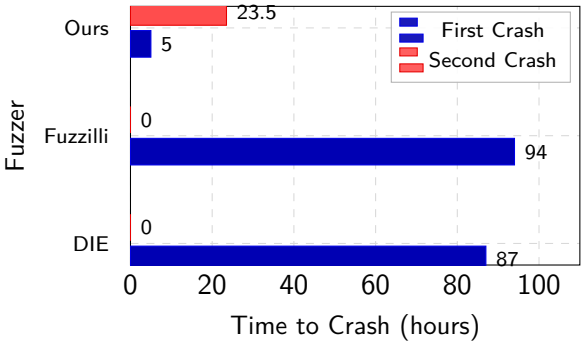
RQ5: Real Bug Discovery

Setup:

- V8 from 2023
- 96-hour campaign
- Compare with:
  - Fuzzilli (NDSS'23)
  - FuzzJIT (USENIX'23)
  - DIE (S&P'20)

Additional Test:

- Latest V8 (2025)
- 24-hour campaign
- Issue #450017974
- P1/S1 priority
- Confirmed and Fixed



Conclusion

In our time-limited runs, proposed method finds crashes faster

RQ6: Runtime Overhead

Setup	Exec/s	Change
Baseline	175	—
All Features	112	-36%
Top 25%	**187**	**+6.9%**

Why Faster?

- Coverage instrumentation expensive
- Avoid redundant checks
- 1.56× faster than all features

Conclusion

Negligible overhead, can improve rate!

Key Insights & Future Directions

Key Insights

Sparse Search Space

Only **25%** of features matter

Dynamic Features Critical

Recall: **28% relative gain** over static with only 5 flags, 31% retention

Synergy

Combined → **3× precision**
Highest overall recall (**0.68**)

Future Work

- Online learning & guided mutations
- Root cause explanation (SHAP)
- Automated crash triaging
- Cross-engine generalization
- Broader domains (kernels, browsers)

Limitations

- V8-specific features
- GPT-4.1 dependency
- Model drifts over time

Concluding Remarks

Central Thesis

The future of fuzzing lies not in exploring **more**, but in exploring **less but smarter** by leveraging historical data.

What We Showed:

- Coverage wastes effort
- Features predict risk
- Small sets suffice
- Data beats heuristics
- Practical & fast

Impact:

- Paradigm shift in fuzzing
- Utilization of historical data
- Reproducible methodology
- Easy to adapt
- Possible applicable beyond JS

Open Science

All data, code, and scripts publicly available

GitHub Repository

`github.com/anon-artifacts/DataCentricFuzzJS`

- 190 curated PoCs
- Feature extraction scripts
- Trained models & data
- Fuzzer implementation
- Experimental results
- Reproduction instructions

Thank You!

Questions and Discussion

Kishan Kumar Ganguly
kgangul@ncsu.edu